

INTRO TO CODING BASICS

Lesson 2: Sequencing — Build a Sandwich

Audience: Primary learners, approximately ages 6–12 **Duration:** 45–60 minutes

Format: Unplugged activity → Coding application → Reflection

PORTFOLIO NOTE: This demonstration expands upon the instructional approach documented in UET's original *Introduction to Coding Basics* proposal. It is a portfolio demonstration of UET's instructional-design capability. It is not represented as an original client deliverable, deployed lesson, or documented learner outcome.

Turning a Familiar Task Into Computational Thinking

Computers follow instructions in a specific order. When the order changes, the result can change. Students discover that **sequencing—the order of instructions—is fundamental to coding.**

1. Lesson at a Glance

Learning Objectives

By the end of the lesson, learners will be able to:

- Define sequencing in their own words.
- Arrange a set of instructions in the correct order.
- Explain why changing the order of instructions can cause an error.
- Create a simple sequence of instructions for a coding task.
- Translate a real-world sequence into a simple block-code sequence.
- Identify and correct an incorrectly ordered sequence.

Skills Focus

Sequencing • Algorithmic thinking • Decomposition • Logical reasoning • Problem-solving • Communication • Collaboration • Debugging

2. Key Vocabulary

Word	Learner-Friendly Meaning
Code	Instructions that tell a computer what to do.
Sequence	The order in which instructions happen.
Algorithm	A step-by-step set of instructions for completing a task.
Instruction	A direction telling someone—or something—what to do.
Debug	Find and fix a mistake in a set of instructions.

Teacher Prompt

“If I give you the right instructions but put them in the wrong order, will the instructions still work?”

3. Materials

Teacher: Printed sequencing cards; board/display; marker; optional sandwich-making materials; computer/tablet; Scratch, Code.org, or another block-based coding environment.

Learners: Sequencing cards; pencil; activity sheet; computer/tablet for the coding portion.

4. Warm-Up — Can You Follow My Instructions?

Time: 5–10 minutes. Use a familiar task such as “**Brush your teeth.**” Ask what happens first, what happens next, and whether the task would work if the steps were rearranged.

Introduce: **SEQUENCE = putting instructions in the order they need to happen.**

5. Learner-Facing Activity — Build a Sandwich

Your Challenge: A computer cannot “just know” what you mean. It needs instructions. Your job is to create the instructions for making a sandwich.

Example Sequencing Cards

CARD D

CARD A Take two slices of bread.	CARD B Spread peanut butter on one slice.	CARD C Spread jelly on the other slice.	Put the slice of bread with peanut butter together with the slice of bread with the jelly, ensuring that the peanut butter and jelly are facing one another.	CARD E Cut the sandwich in half.
--	---	---	--	--

Step 1 — Look at the Cards: Your teacher gives you cards containing sandwich-making instructions.

Step 2 — Put Them in Order: Work with your partner. Arrange the cards so someone could follow them from beginning to end. Write the order: 1. ____ 2. ____ 3. ____ 4. ____ 5. ____

Step 3 — Become the Computer: Your partner gives you the instructions one step at a time. Do only what the instruction tells you to do. Do not skip ahead.

6. Think Like a Coder

Change one card so the sequence is incorrect—for example, put “Cut the sandwich in half” before the sandwich has been assembled.

Ask: Which step is in the wrong place? Where should it go? How do you know? What would happen if a computer followed the incorrect sequence exactly?

Key Concept: A computer does not “fix” your instructions for you. The instructions need to be clear and correctly ordered.

7. Guided Coding Application

Time: 15–20 minutes. The sandwich instructions were an algorithm. Coding lets us turn instructions into commands that a computer can follow.

Coding Challenge: Create a simple animation in which a character performs three actions in order.

Example: **When program starts** → **Move** → **Say “Hello!”** → **Move again** → **Say “I am learning to code!”**

Challenge 1 — Create a Sequence: Construct a short sequence using available blocks.

Challenge 2 — Test It: Run the program. Did the character do everything in the correct order?

Challenge 3 — Change the Order: Move one block and run the program again. Observe what changed.

Challenge 4 — Fix It: Return the blocks to the intended sequence and test again.

8. Independent Learner Challenge — Code My Morning

Create a sequence of **at least five steps** showing how a character completes a familiar morning task: getting ready for school, preparing breakfast, getting dressed, packing a school bag, or getting ready for a journey.

Your algorithm must: ■ Have a clear beginning ■ Contain at least five steps ■ Put the steps in logical order ■ Include a character or object ■ Be tested ■ Be corrected if something does not work.

Bonus: Make your character say something between two of the steps.

9. Teacher / Instructor Guidance

Before the Lesson: Prepare a sequencing-card set for each pair/group, an incorrectly ordered sequence, access to the coding platform, and a short coding challenge appropriate to learner experience.

During the Unplugged Activity: Do not immediately tell learners the correct answer. Ask: “What should happen first?” “How do you know?” “What would happen if we did this step first?” “Would the result still work?” “Can you explain your reasoning?”

The objective is not simply to produce the correct sequence. The objective is for learners to **reason about why the sequence works**.

During Coding: Emphasize **Plan** → **Build** → **Test** → **Fix**. When a sequence does not work, ask: “Which instruction happened too early?” or “Which instruction should happen before this one?”

10. Differentiation

More Support: Fewer sequencing cards • picture-based instructions • numbered spaces • partially completed sequence • partner support.

More Challenge: Create a longer algorithm • include a deliberate error for a partner to find • create two sequences and compare them • explain why one works • translate the algorithm into a coding project.

11. Reflection & Assessment

Exit Ticket

1. Sequencing means: _____
2. If I put instructions in the wrong order: _____
3. One mistake I found today was: _____
4. I fixed it by: _____
5. One thing I now understand about coding is: _____

12. Quick Formative Assessment

Success Criteria	Yes	Almost	Not Yet
Explain sequencing	■	■	■
Arrange steps logically	■	■	■
Explain why order matters	■	■	■
Build a simple sequence	■	■	■
Test the sequence	■	■	■
Identify an error	■	■	■
Correct the sequence	■	■	■

Evidence of Learning

Completed sequencing activity • verbal explanation • correctly ordered algorithm • working coding sequence • corrected coding error • exit-ticket response.

13. Closing Discussion

Ask: “What does making a sandwich have to do with coding?” Guide learners toward: **Everyday task** → **Step-by-step instructions** → **Sequence** → **Algorithm** → **Code** → **Computer follows instructions**.

Then ask where else sequences are used in everyday life: cooking, getting dressed, playing a game, following directions, building something, using an app, or completing schoolwork.

14. Instructor Takeaway

LEVEL 1 — EXPERIENCE: Learners physically manipulate familiar steps. **Build a Sandwich.**

LEVEL 2 — CONCEPT: Learners identify **Sequencing + Algorithms**.

LEVEL 3 — APPLICATION: Learners translate the concept into **Block-Based Coding**.

This progression makes an abstract programming concept concrete before asking learners to work with code.

15. UET Instructional Design Signature

- Start Unplugged** — Introduce concepts through physical and relatable activities.
- Make It Relevant** — Connect coding concepts to familiar experiences.
- Use Visual Coding** — Move from physical instructions into drag-and-drop programming.
- Encourage Collaboration** — Use partners to discuss, construct, test, and solve.
- Reinforce Through Application** — Move from explanation → activity → coding → reflection.
- Build Computational Thinking** — Have learners decompose tasks, design algorithms, test solutions, and correct errors.

16. Portfolio Evidence Statement

This artifact demonstrates UET’s ability to translate a coding concept into age-appropriate instruction; design an unplugged-to-digital learning progression; create learner-facing activities and instructor guidance; structure measurable learning objectives; incorporate computational-thinking skills; and integrate testing, debugging, and reflection.

17. Provenance & Portfolio Boundary

Source foundation	UET’s original Introduction to Coding Basics proposal
Portfolio asset	UET portfolio demonstration expanding upon the documented instructional approach
Original client deliverable	No
Claimed implementation	None
Claimed learner outcomes	None
Purpose	Demonstrate UET instructional-design and curriculum-development capability

UJAMAA EDUCATIONAL TECHNOLOGIES

Technology • AI • Learning • Entrepreneurship

Turning knowledge into opportunity.